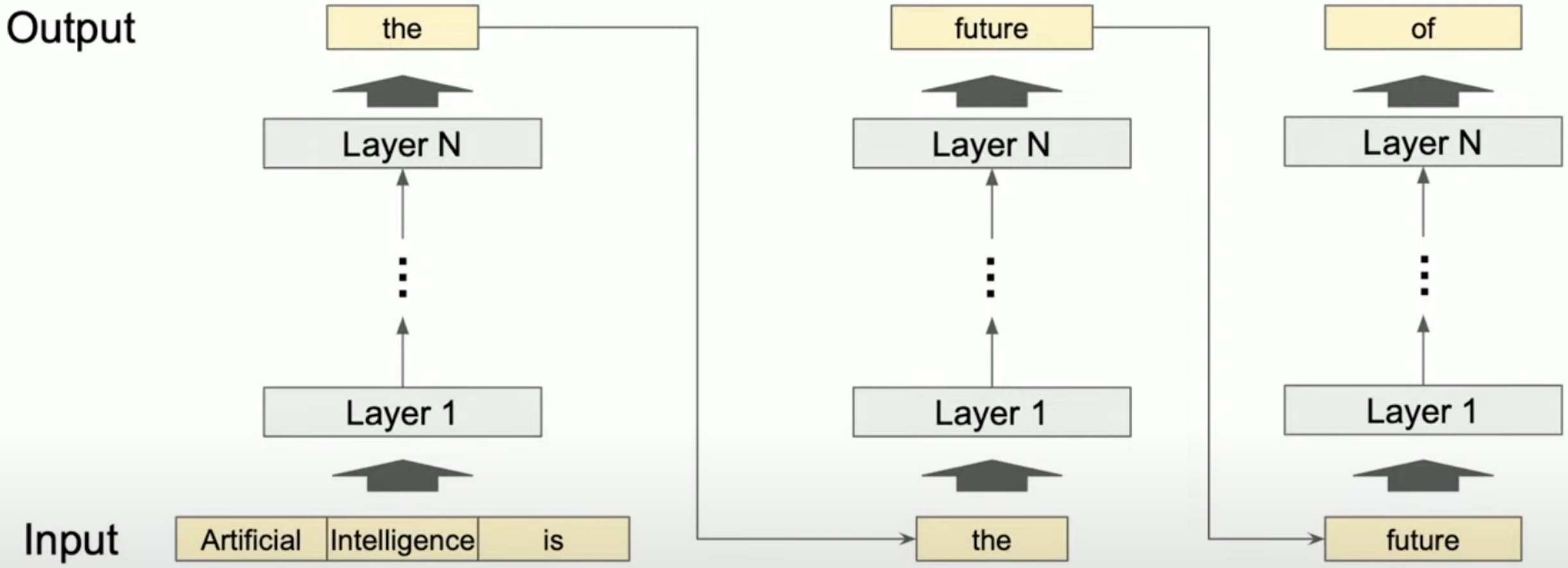
PagedAttention

Danyang Zhuo

Duke University

Some materials are from vLLM's Ray Summit Slides and Sarathi-Serve's OSDI Slides

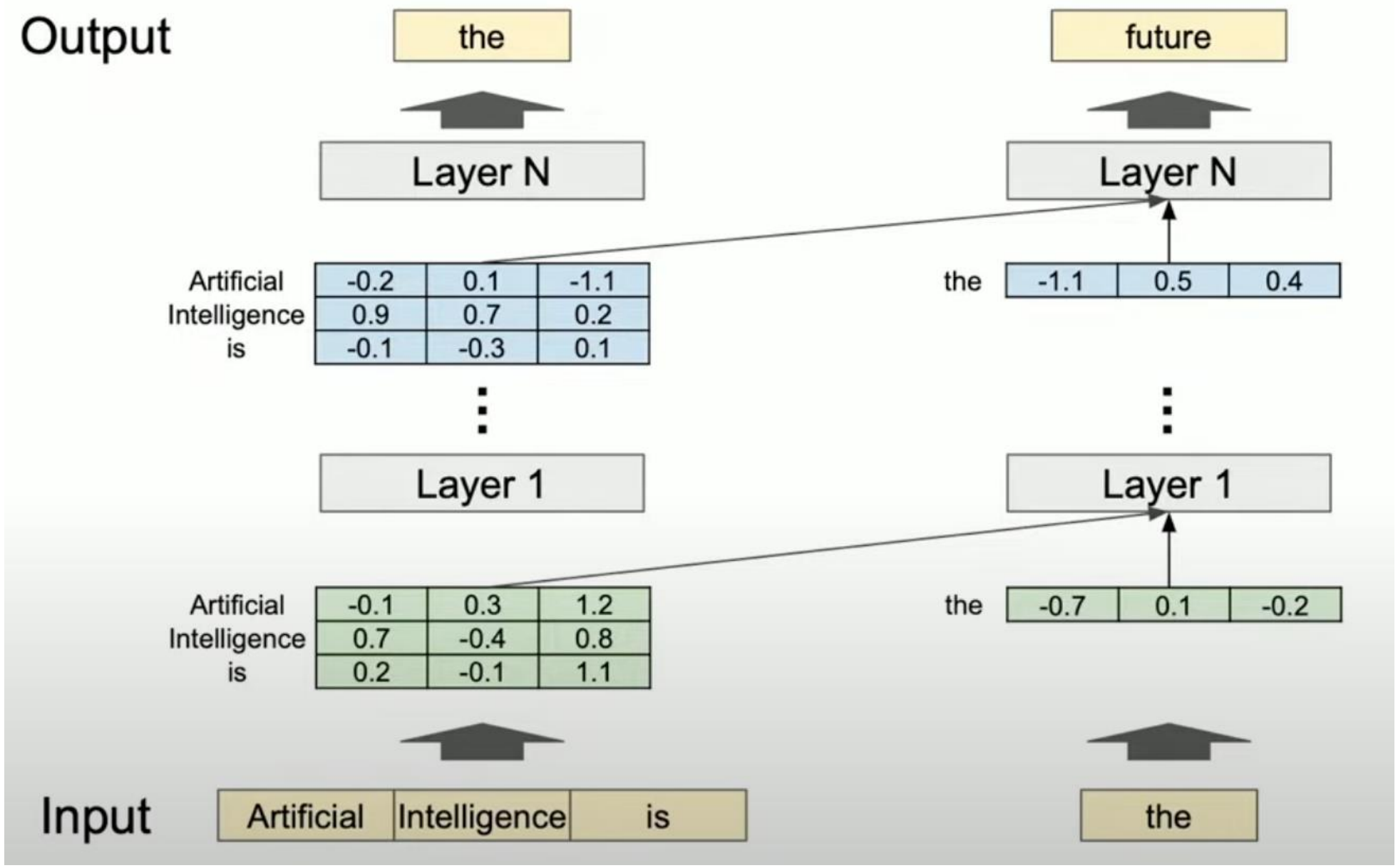
Inference process of LLMs



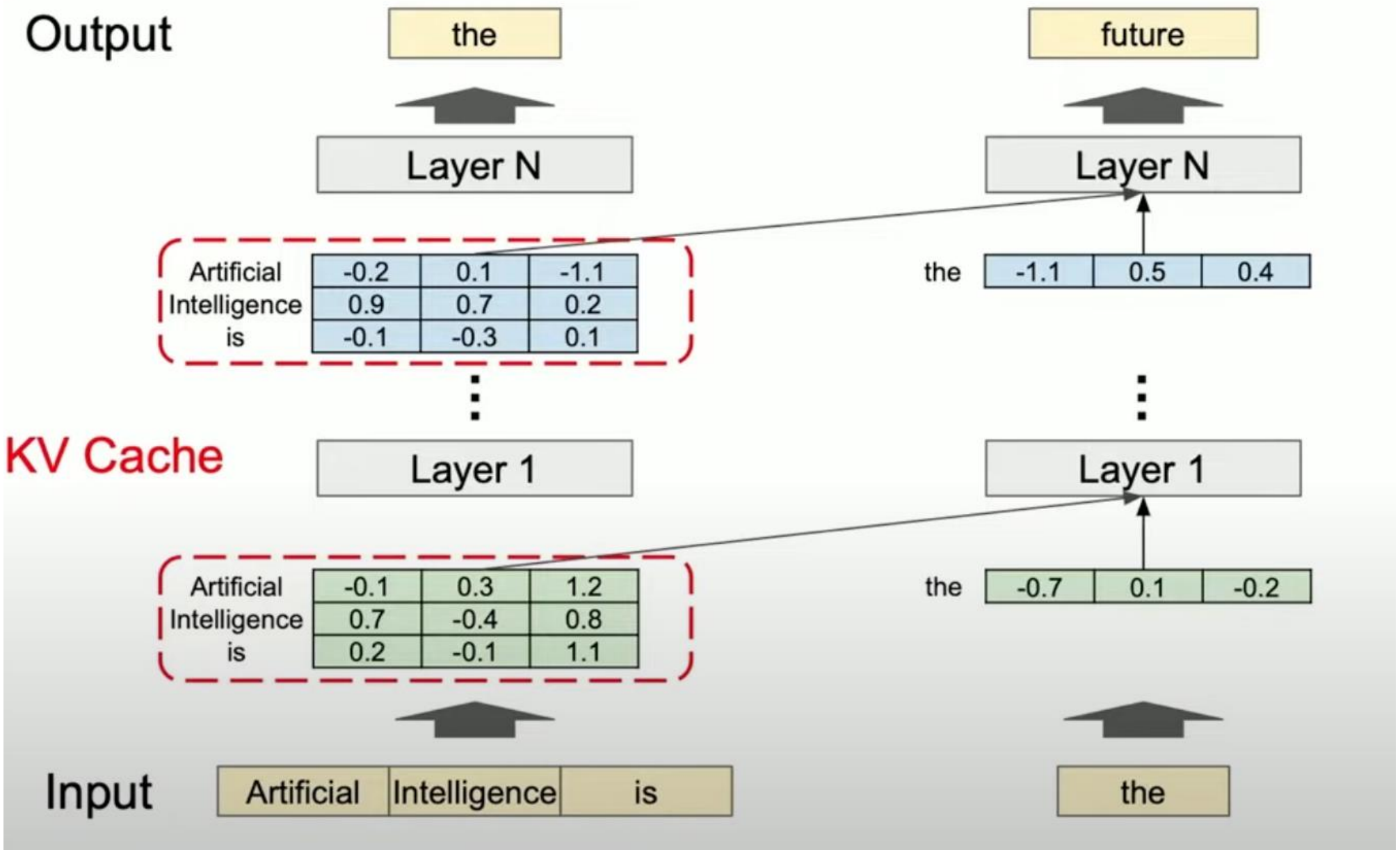
Repeat until the sequence

- Reaches its pre-defined maximum length (e.g., 2048 tokens)
- Generates certain tokens (e.g., "<|*end of sequence*|>")

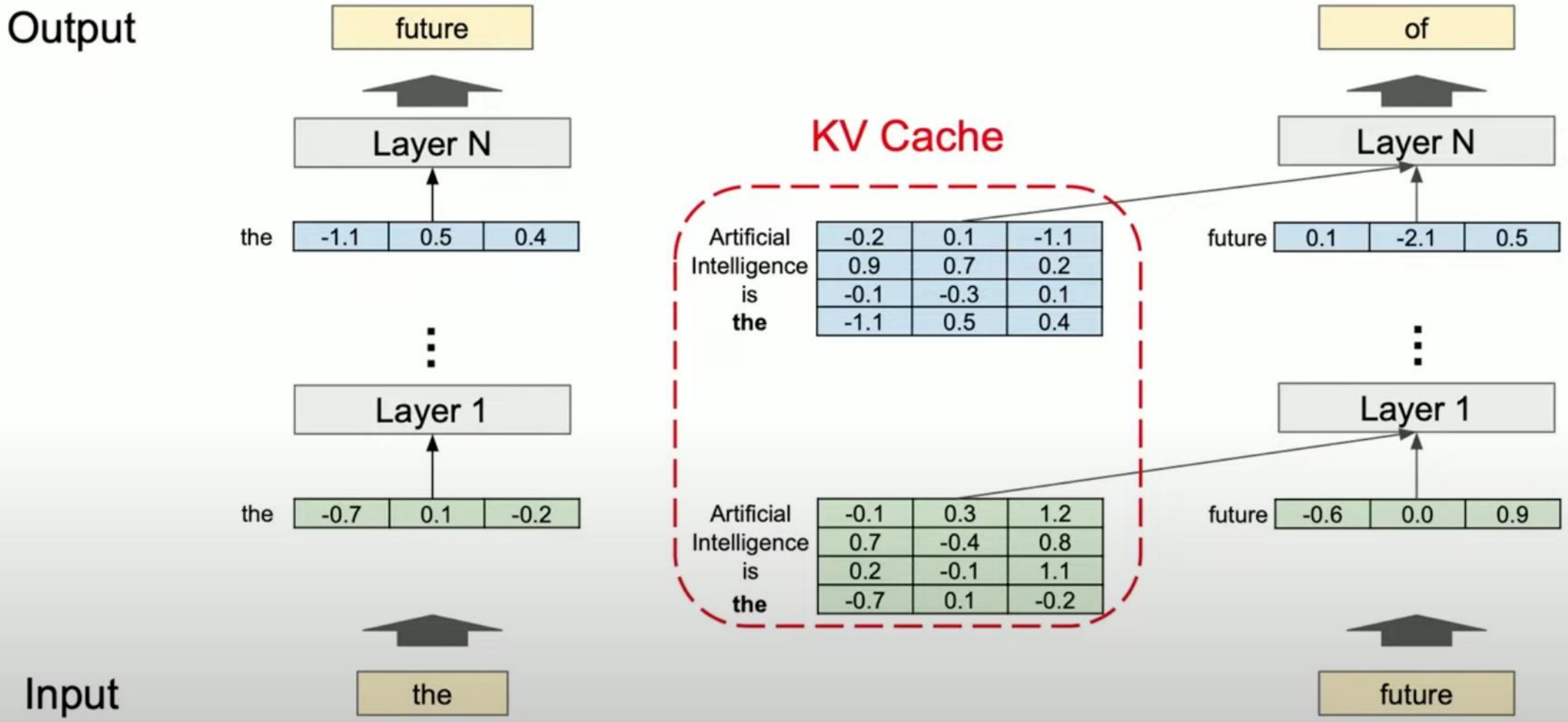
KV Cache


Output
the
future
Layer N
Layer N
Artificial
Intelligence
is
-0.2
0.1
-1.1
0.9
0.7
0.2
-0.1
-0.3
0.1
the
-1.1
0.5
0.4
Layer 1
Layer 1
Artificial
Intelligence
is
-0.1
0.3
1.2
0.7
-0.4
0.8
0.2
-0.1
1.1
the
-0.7
0.1
-0.2
Input
Artificial
Intelligence
is
the

KV Cache

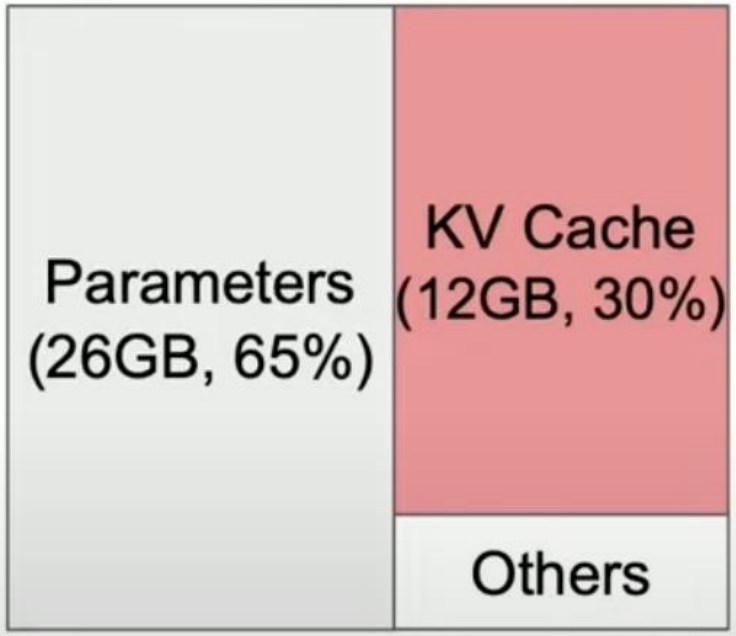

Output
the
future
Layer N
Layer N
Artificial
Intelligence
is
-0.2
0.1
-1.1
0.9
0.7
0.2
-0.1
-0.3
0.1
the
-1.1
0.5
0.4
KV Cache
Layer 1
Layer 1
Artificial
Intelligence
is
-0.1
0.3
1.2
0.7
-0.4
0.8
0.2
-0.1
1.1
the
-0.7
0.1
-0.2
Input
Artificial
Intelligence
is
the

KV Cache

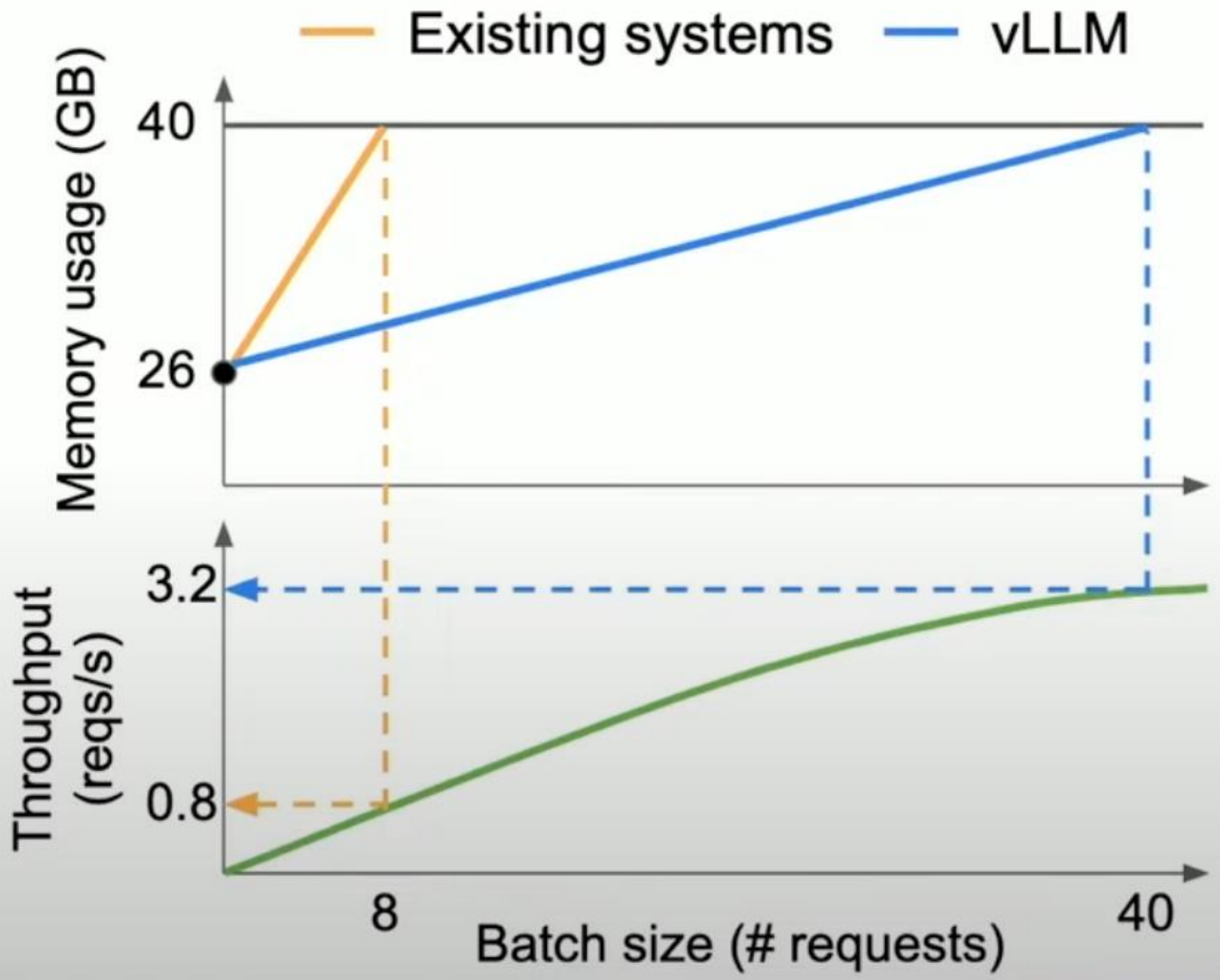

Output
future
of
Layer N
Layer N
KV Cache
the
-1.1
0.5
0.4
Artificial
Intelligence
is
the
-0.2
0.1
-1.1
0.9
0.7
0.2
-0.1
-0.3
0.1
-1.1
0.5
0.4
future
0.1
-2.1
0.5
Layer 1
Layer 1
the
-0.7
0.1
-0.2
Artificial
Intelligence
is
the
-0.1
0.3
1.2
0.7
-0.4
0.8
0.2
-0.1
1.1
-0.7
0.1
-0.2
future
-0.6
0.0
0.9
Input
the
future

Key insight

Efficient management of KV cache is crucial for high-throughput LLM serving



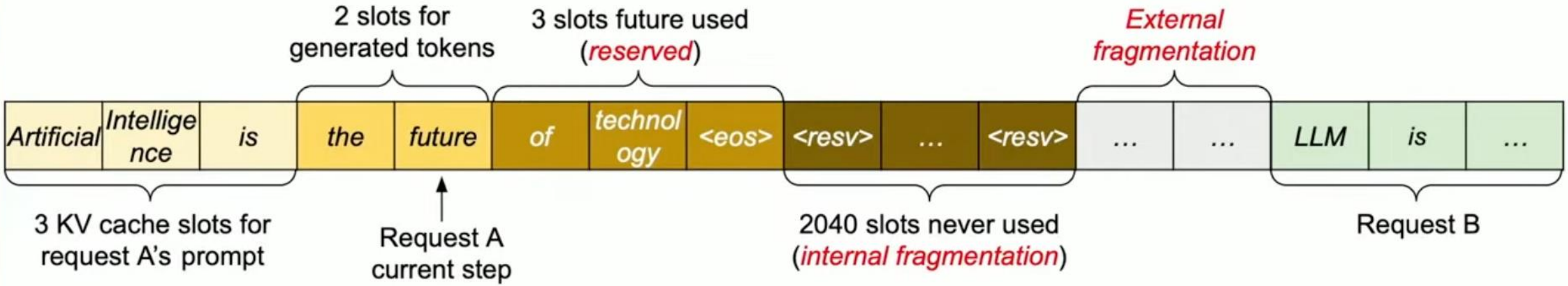
13B LLM on A100-40GB



How much a request needs?

- For a 13B parameter OPT model, the KV cache of a single token demands 800 KB of space, calculated as 2 (key and value vectors) × 5120 (hidden state size) × 40 (number of layers) × 2 (bytes per FP16). Since OPT can generate sequences up to 2048 tokens, the memory required to store the KV cache of one request can be as much as 1.6 GB.

Memory waste in KV Cache



- Internal fragmentation: over-allocated due to the unknown output length.
- Reservation: not used at the current step, but used in the future
- External fragmentation: due to different sequence lengths.

Only **20–40%** of KV cache is utilized to store token states

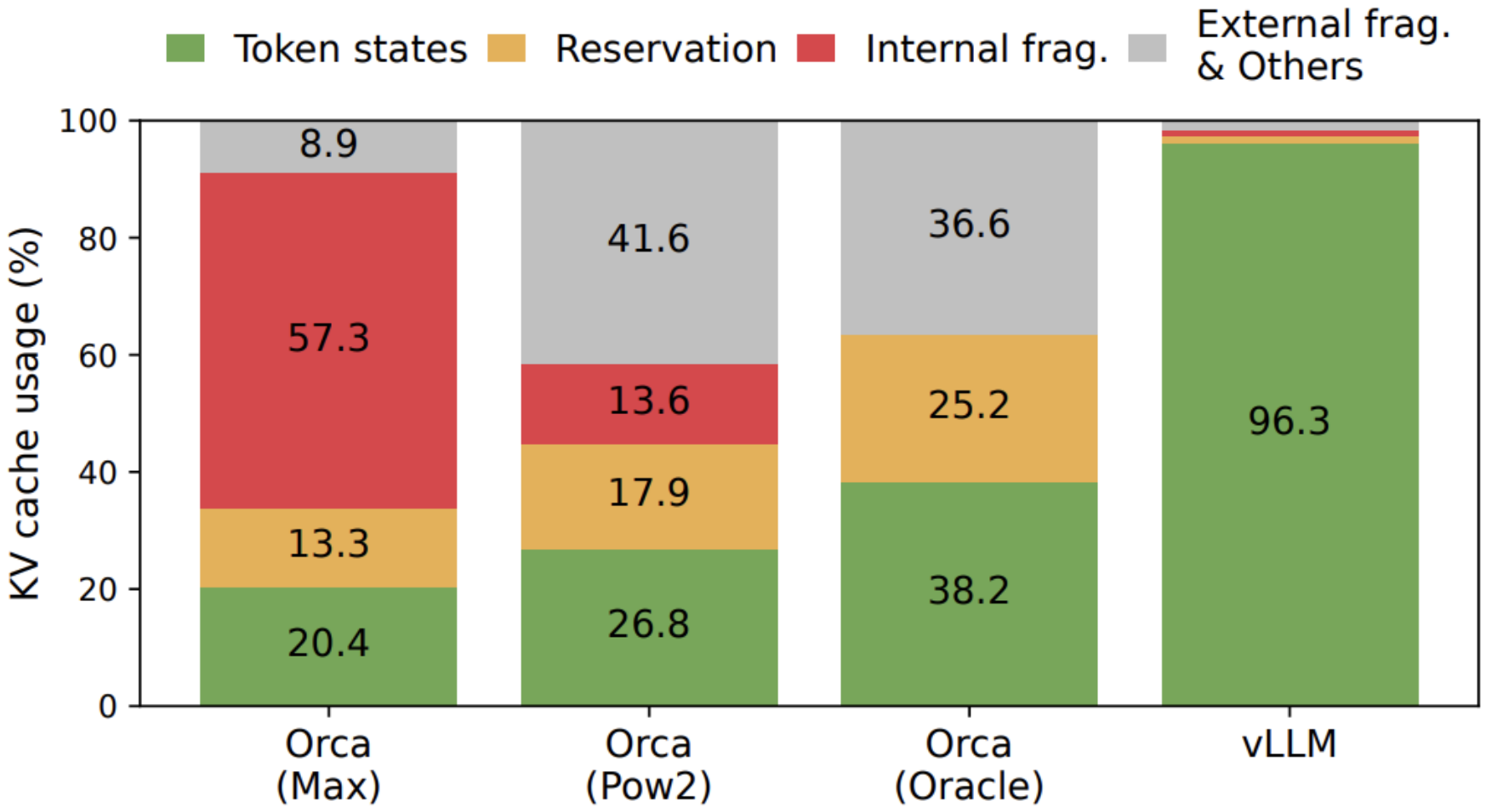


Figure 2. Average percentage of memory wastes in different LLM serving systems during the experiment in §6.2.

Memory challenges in LLM inference

- Large KV cache
- Complex decoding algorithms
  - Prefix sharing
- Unknown input/output length

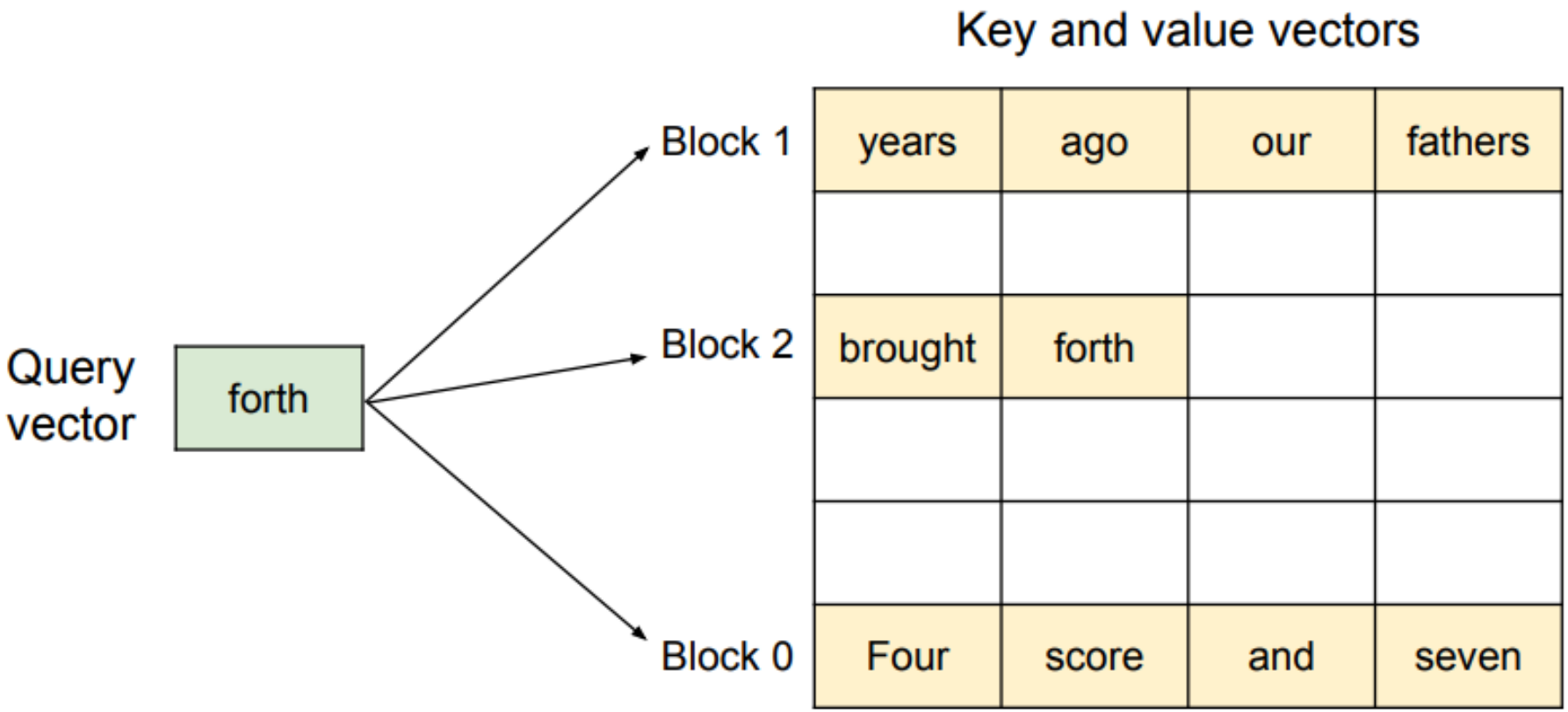


Figure 5. Illustration of the PagedAttention algorithm, where the attention key and values vectors are stored as non-contiguous blocks in the memory.

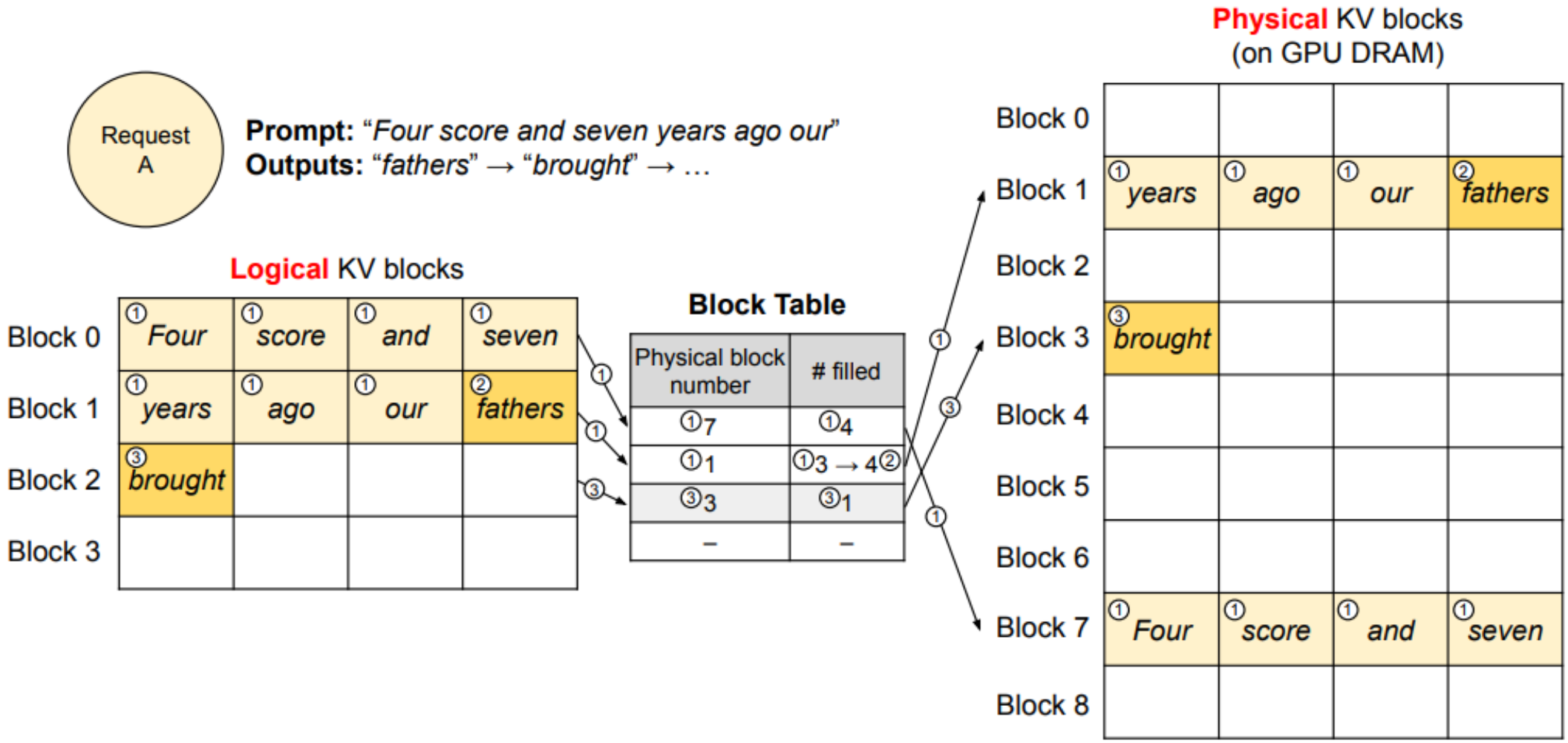


Figure 6. Block table translation in vLLM.

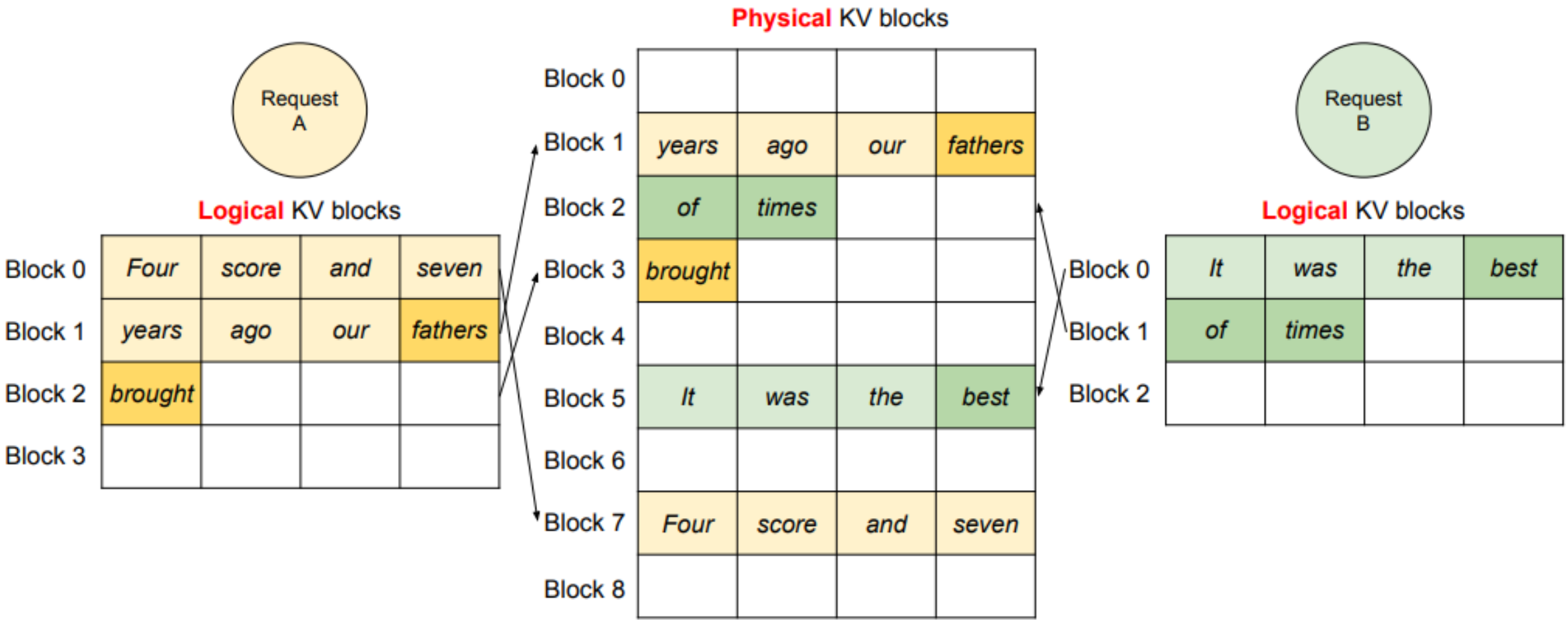


Figure 7. Storing the KV cache of two requests at the same time in vLLM.

Memory efficiency of vLLM

- Minimal internal fragmentation
  - Only happens at the last block of a sequence
  - **# wasted tokens / seq < block size**
    - Sequence: O(100) – O(1000) tokens
    - Block size: 16 or 32 tokens
- No external fragmentation

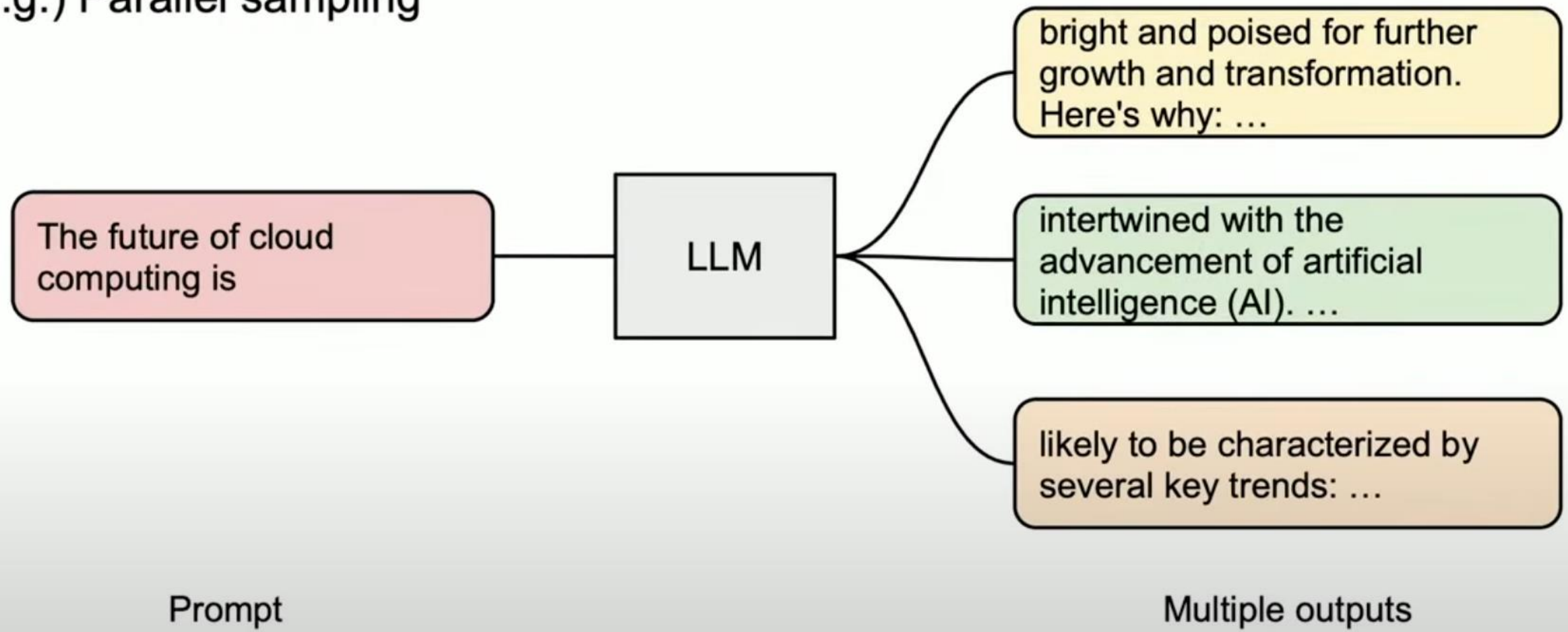
Alan	Turing	is	a
computer	scientist	and	mathematic ian
renowned			

Internal fragmentation

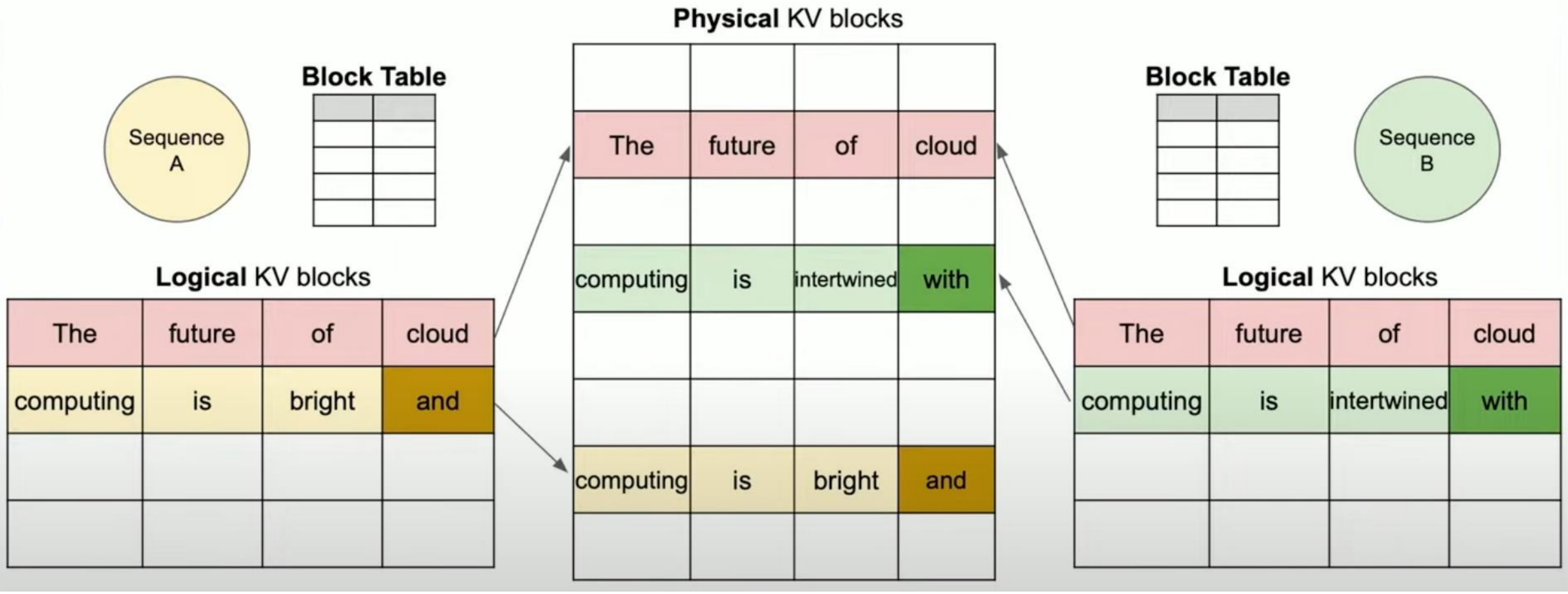
On average, the wasted space is less than **4%** of the KV cache space (**3-5x** improved memory utilization)

Dynamic block mapping enables sharing

E.g.) Parallel sampling



Sharing KV blocks



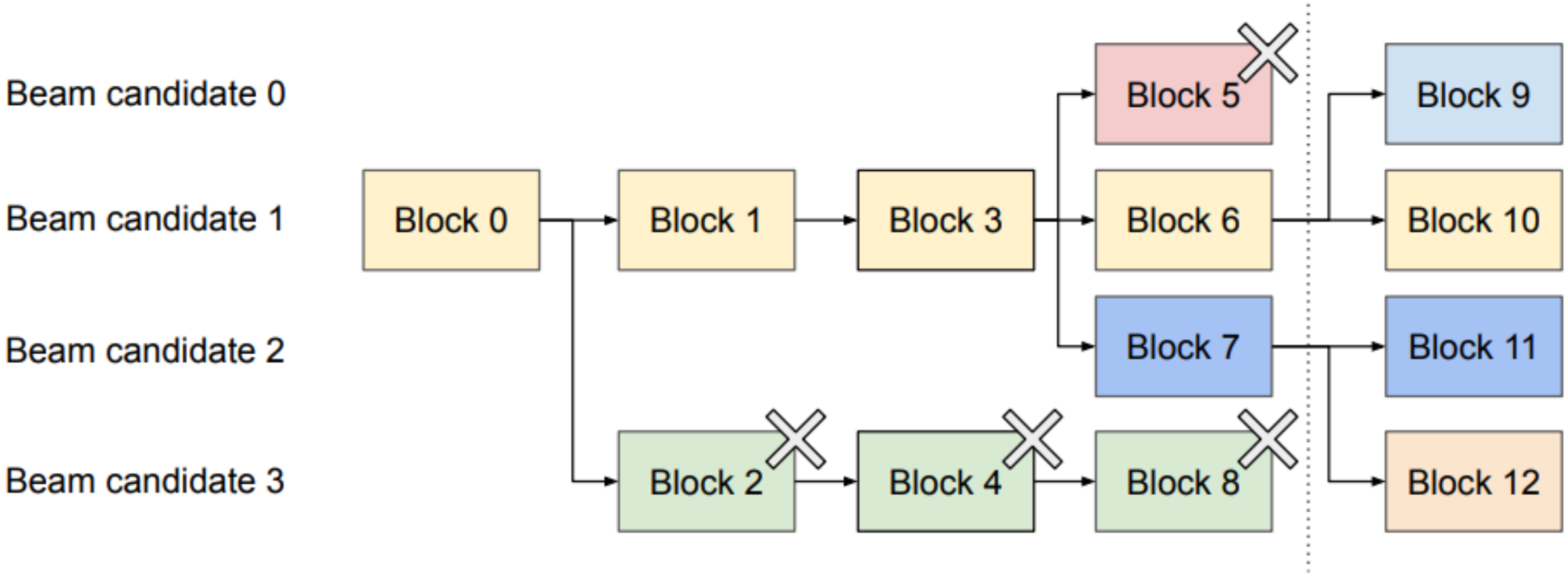


Figure 9. Beam search example.

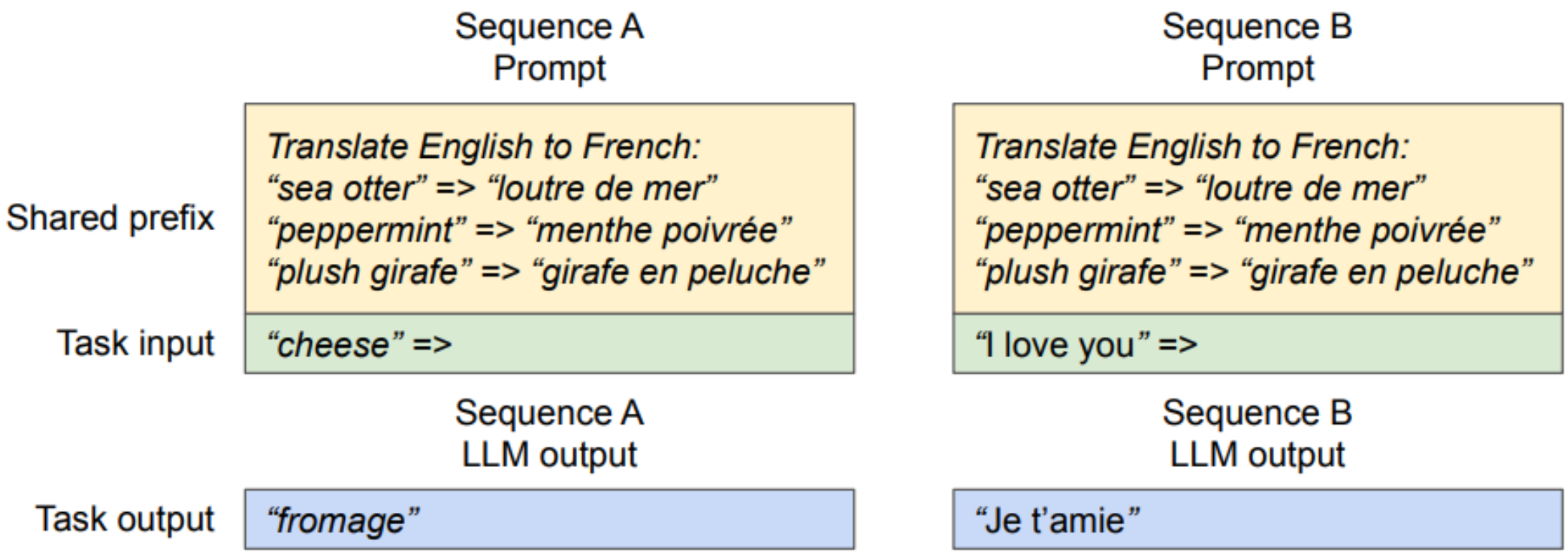


Figure 10. Shared prompt example for machine translation. The examples are adopted from [5].

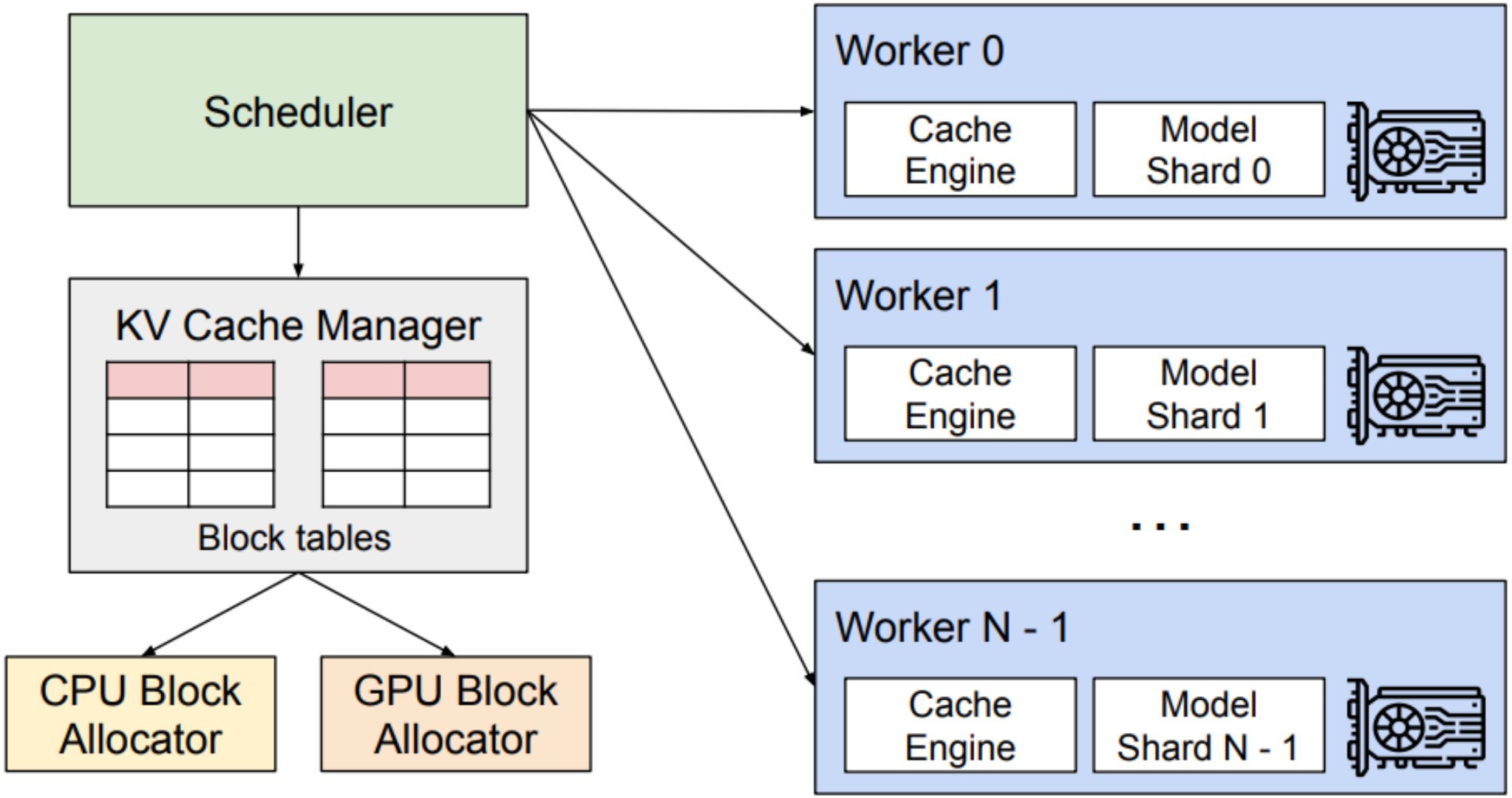


Figure 4. vLLM system overview.

Preemption

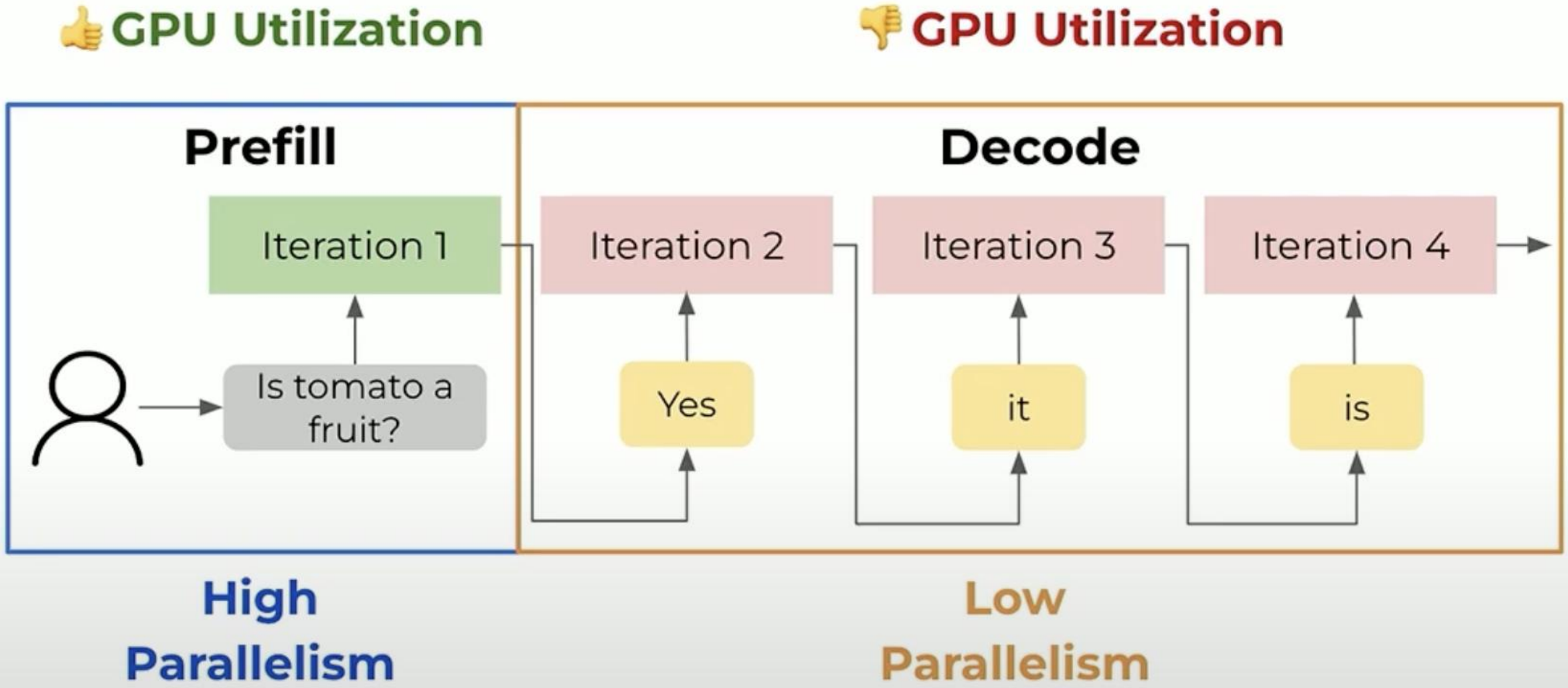
- Swapping
  - Select sequences and evict to CPU memory
- Re-computation
  - Treat decoded tokens as prefill tokens

Taming Throughput-Latency Tradeoff in LLM Inference with *Sarathi-Serve*

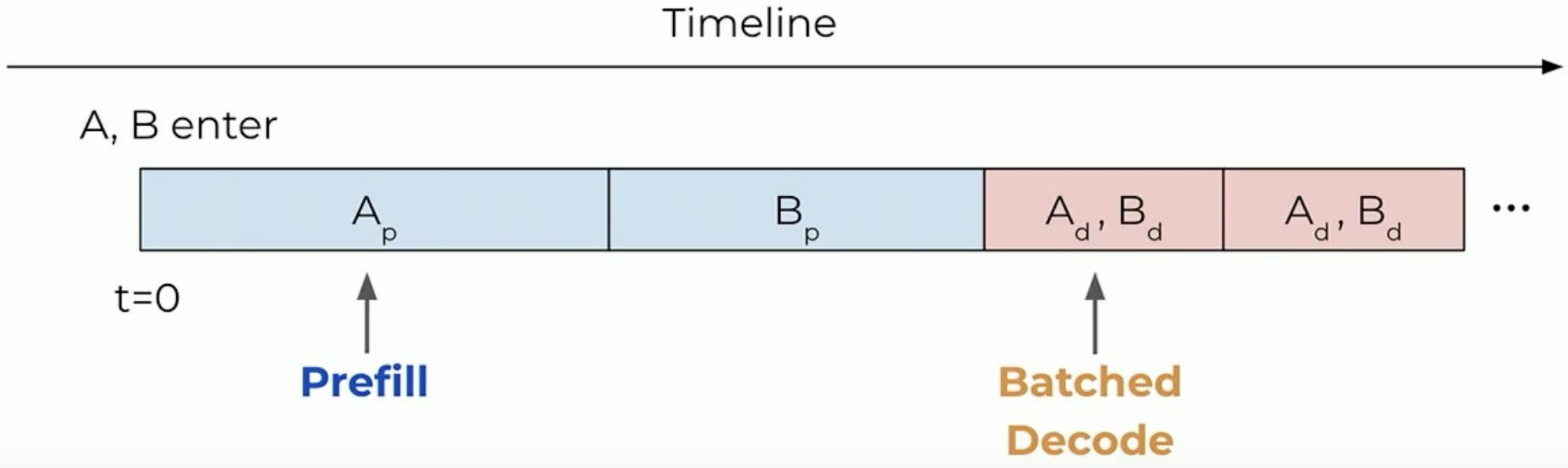
Amey Agrawal*[2], Nitin Kedia[1], Ashish Panwar[1], Jayashree Mohan[1], Nipun Kwatra[1],
Bhargav S. Gulavani[1], Alexey Tumanov[2], and Ramachandran Ramjee[1]

[1]Microsoft Research India
[2]Georgia Institute of Technology

🔄 Background: LLM Inference 101



🥞 Background: Batching LLM Inference



Decode efficiency increases linearly with batch size 🚀

🔼 **Batch size** ⇒ 🔼 **Throughput**

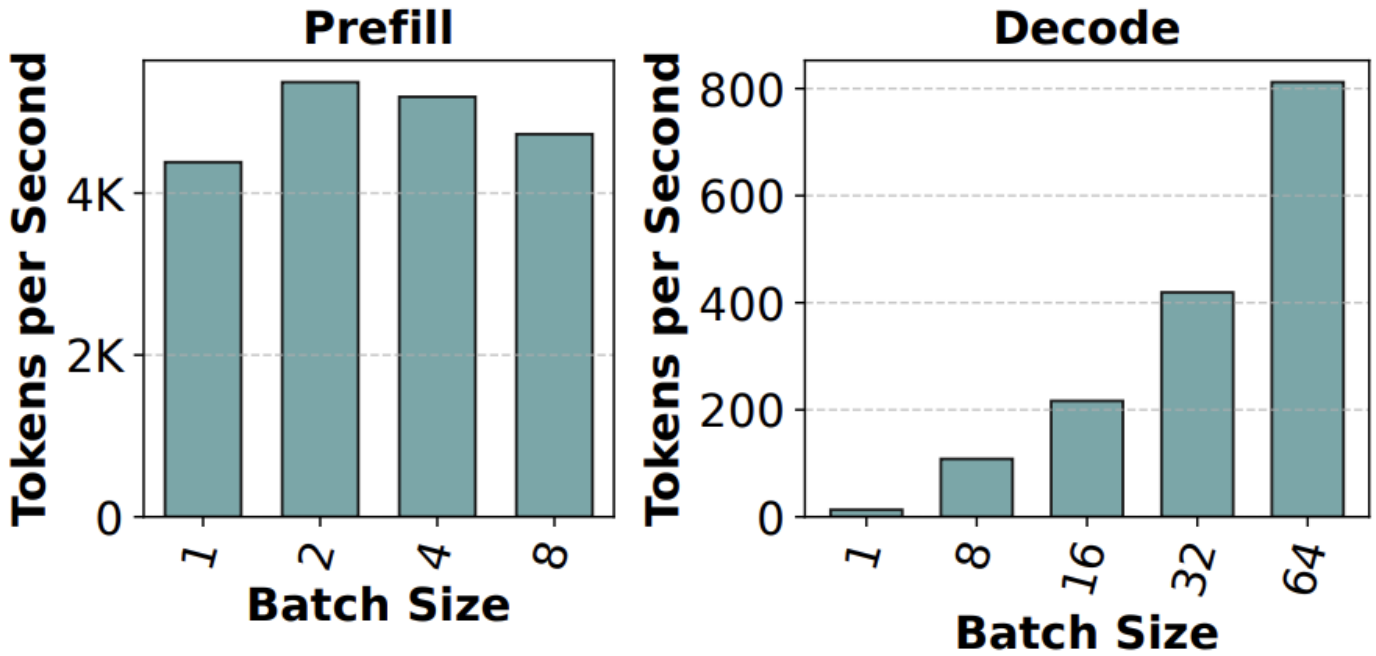
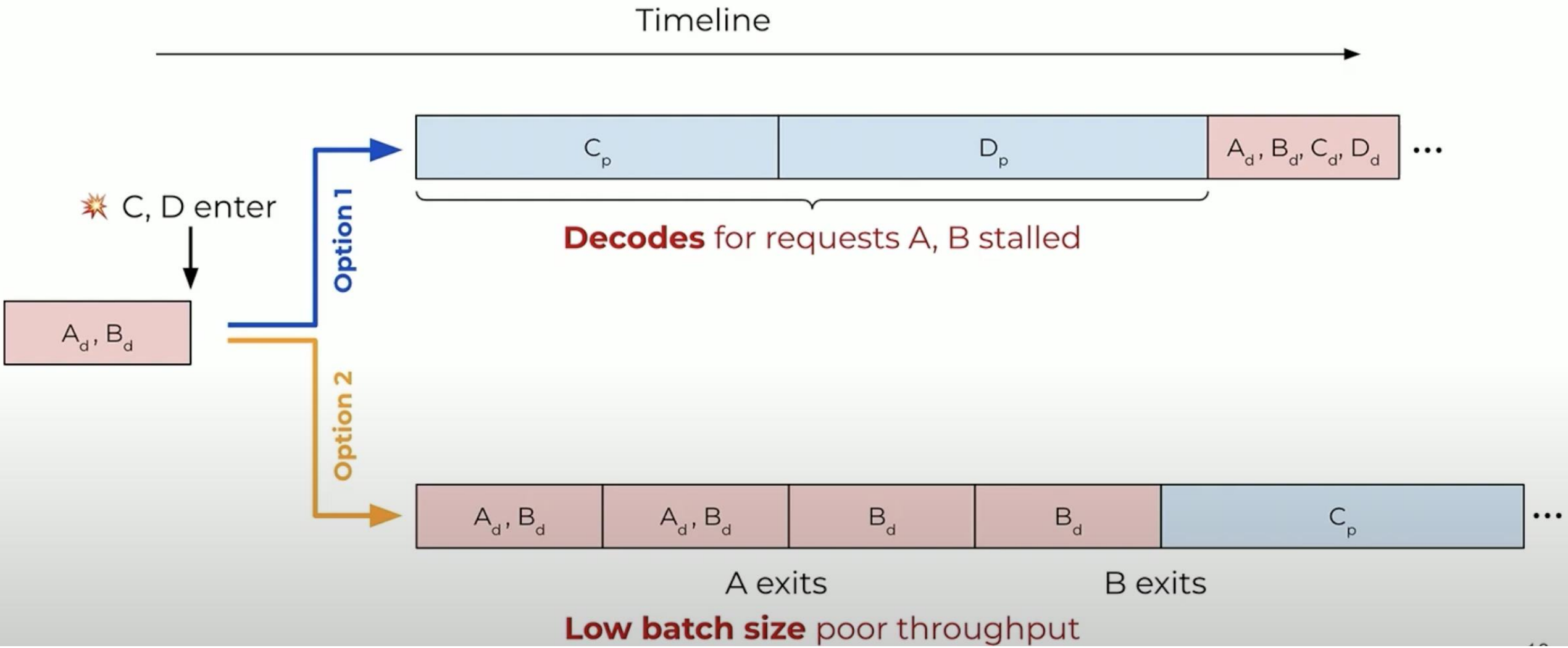


Figure 3: Throughput of the prefill and decode phases with different batch sizes for Mistral-7B running on a single A100 GPU. We use prompt length of 1024 for both prefill and decode experiments. Note that different y-axis, showing prefills are much more efficient than decode. Further, note that *batching boosts decode throughput almost linearly but has a marginal effect on prefill throughput.*

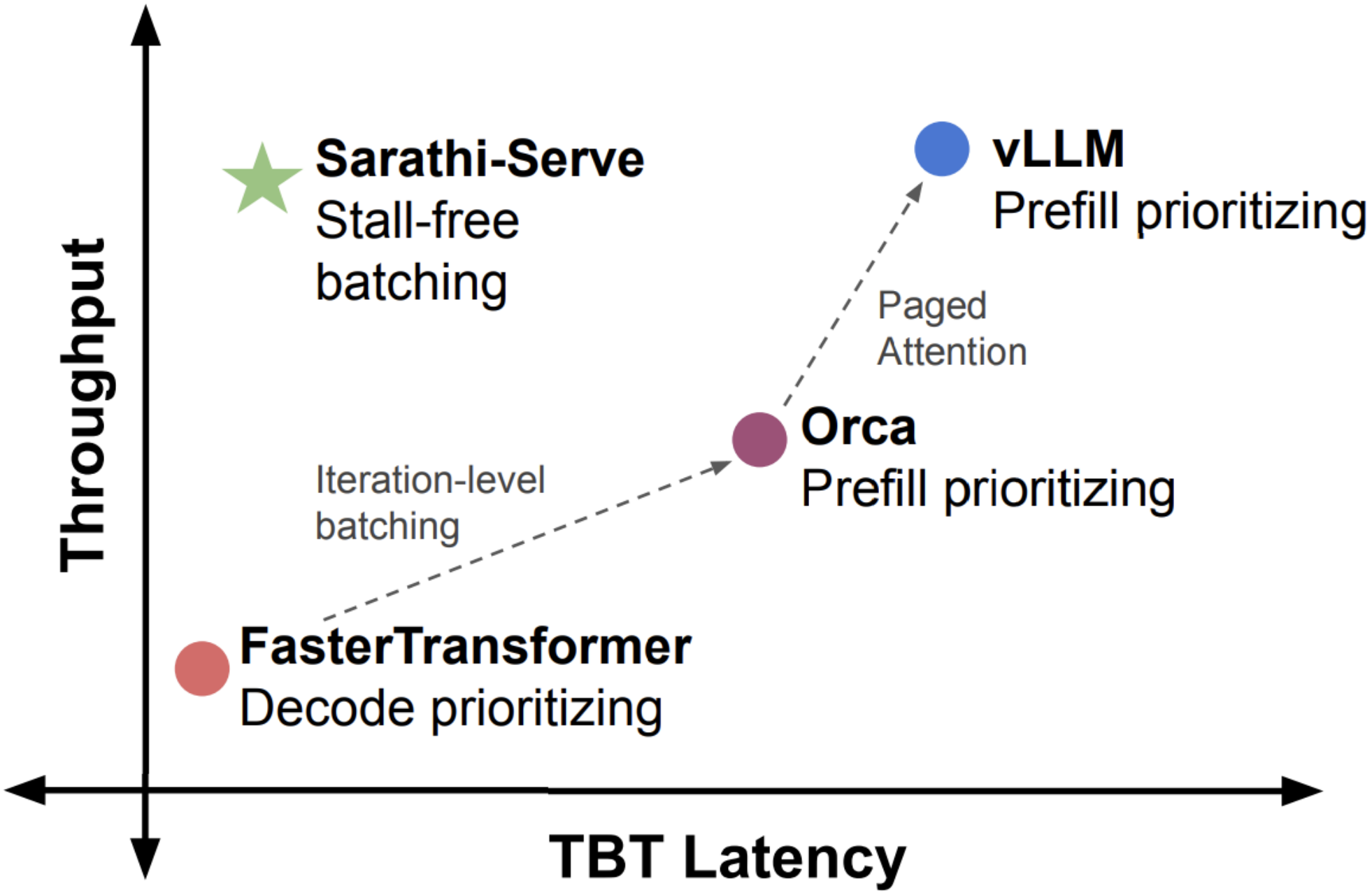
🔎 The Prefill-Decode Scheduling Conundrum



Prefill vs decode

- Decode prioritization
  - Scheduler issues a batch of requests
  - These requests finish their “decode” before any request can “prefill”

- Prefill prioritization
  - Scheduler issues a batch of iterations, mixed of prefill and decode
  - “Prefill” of new requests take a long time and will block “decode” for existing requests


Sarathi-Serve
Stall-free
batching
vLLM
Prefill prioritizing
Paged
Attention
Orca
Prefill prioritizing
Iteration-level
batching
FasterTransformer
Decode prioritizing
Throughput
TBT Latency

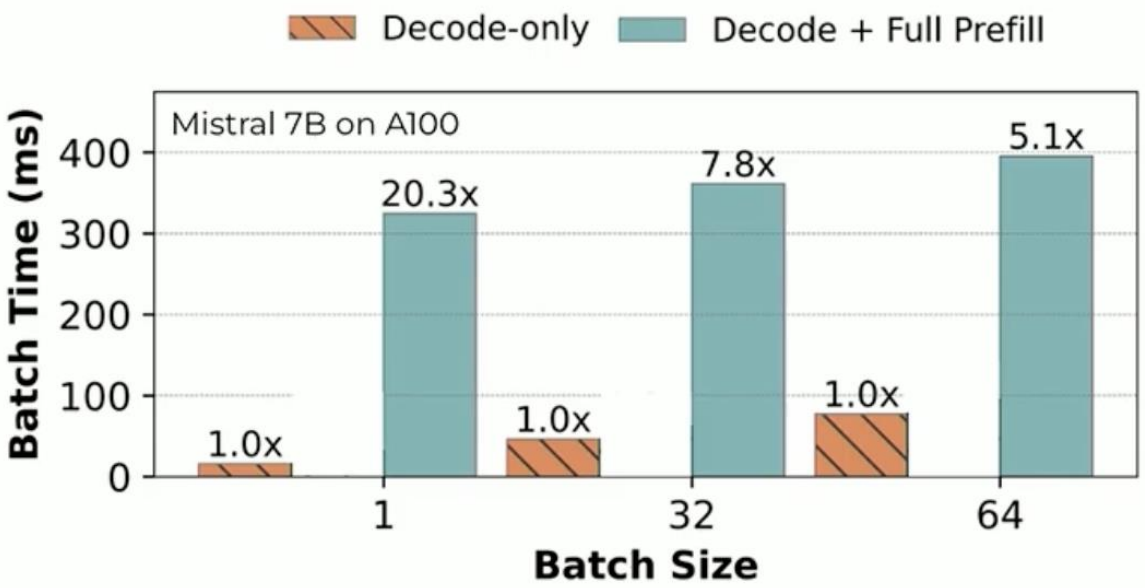
🔎 Mixed Batching

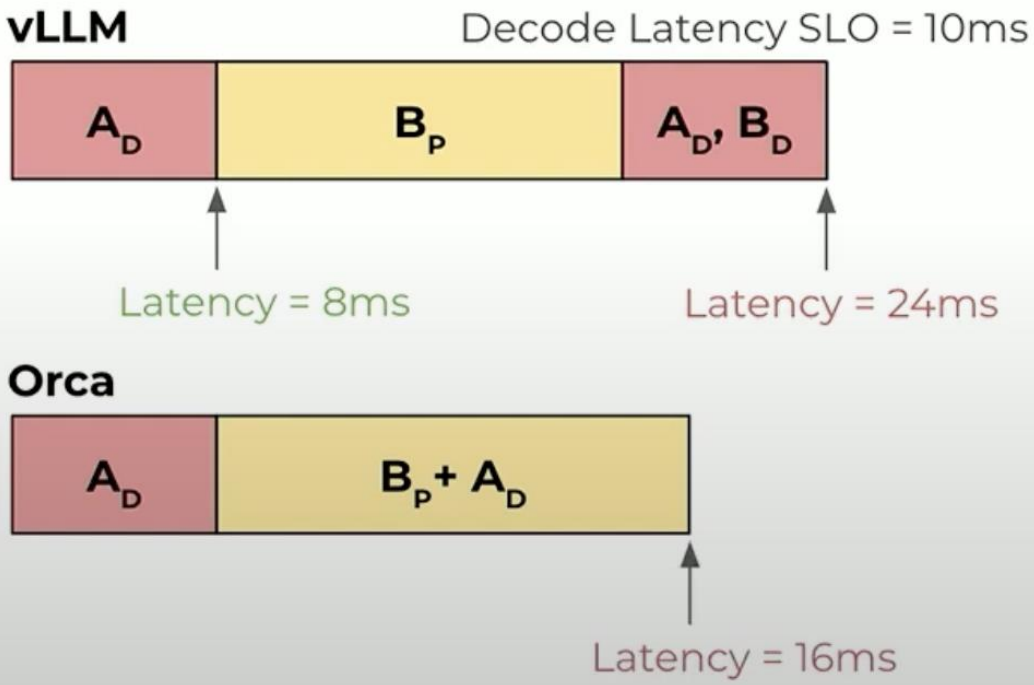
Idea

👮 Fused computation of prefill and decodes

Challenge

😭 Naively combining prefill and decode operations leads to increase in latency





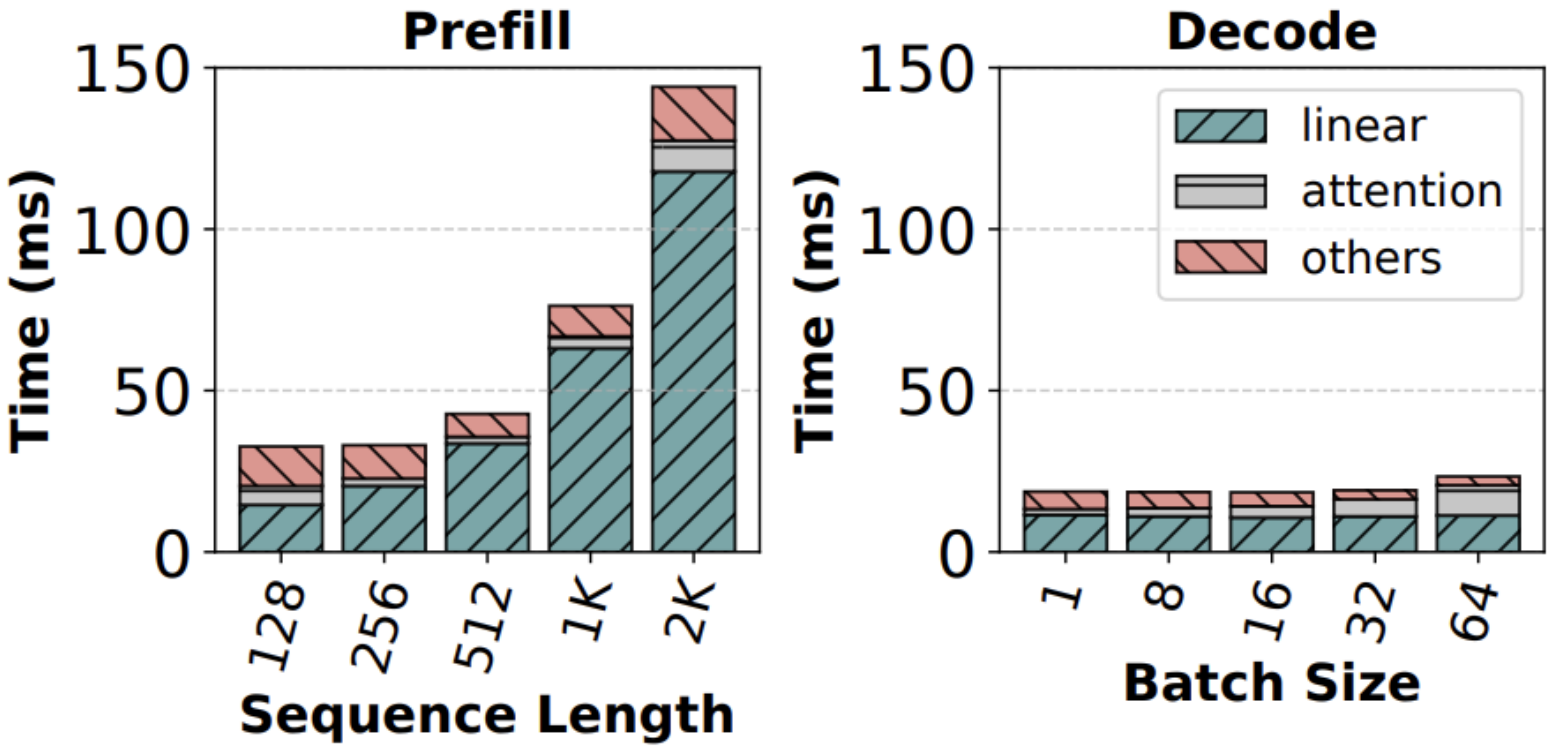


Figure 4: Prefill and decode time with different input sizes for Mistral-7B running on single A100 GPU. Linear layers contribute to the majority of runtime in both prefill and decode phases. Due to the low arithmetic intensity in decode batches, the cost of linear operation for 1 decode token is nearly same as 128 prefill tokens.

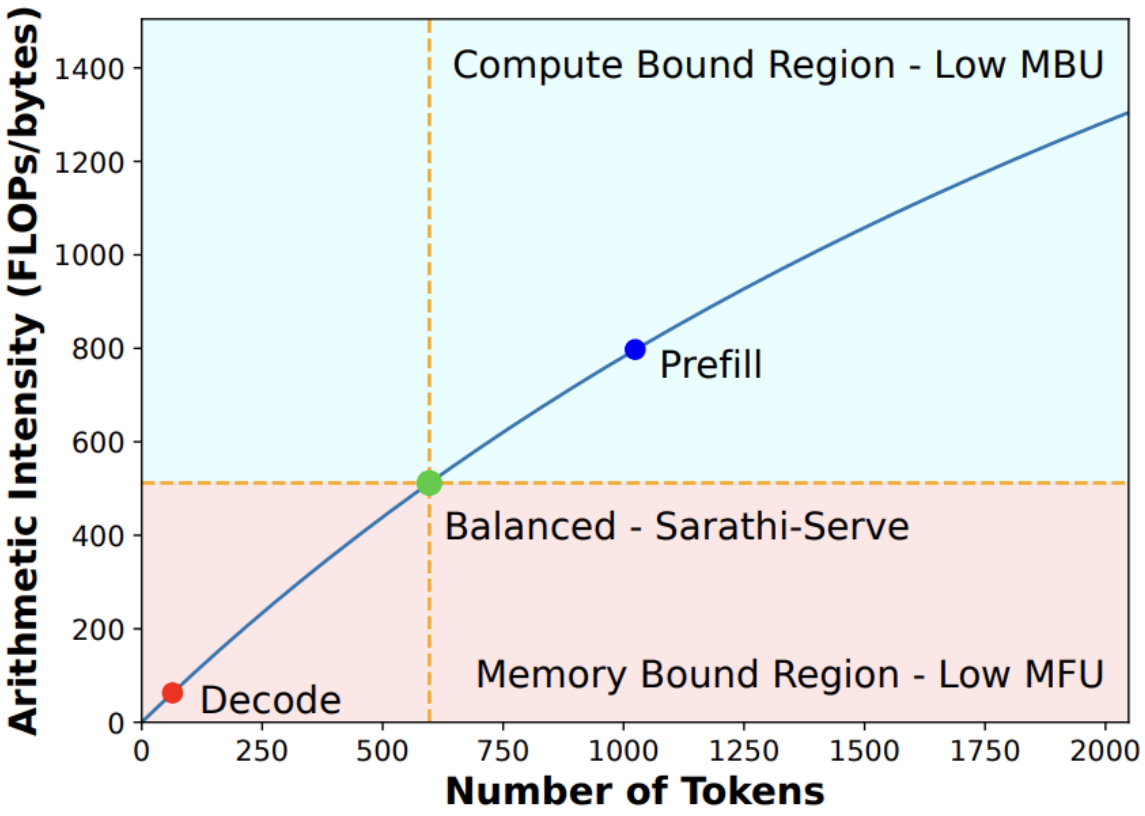
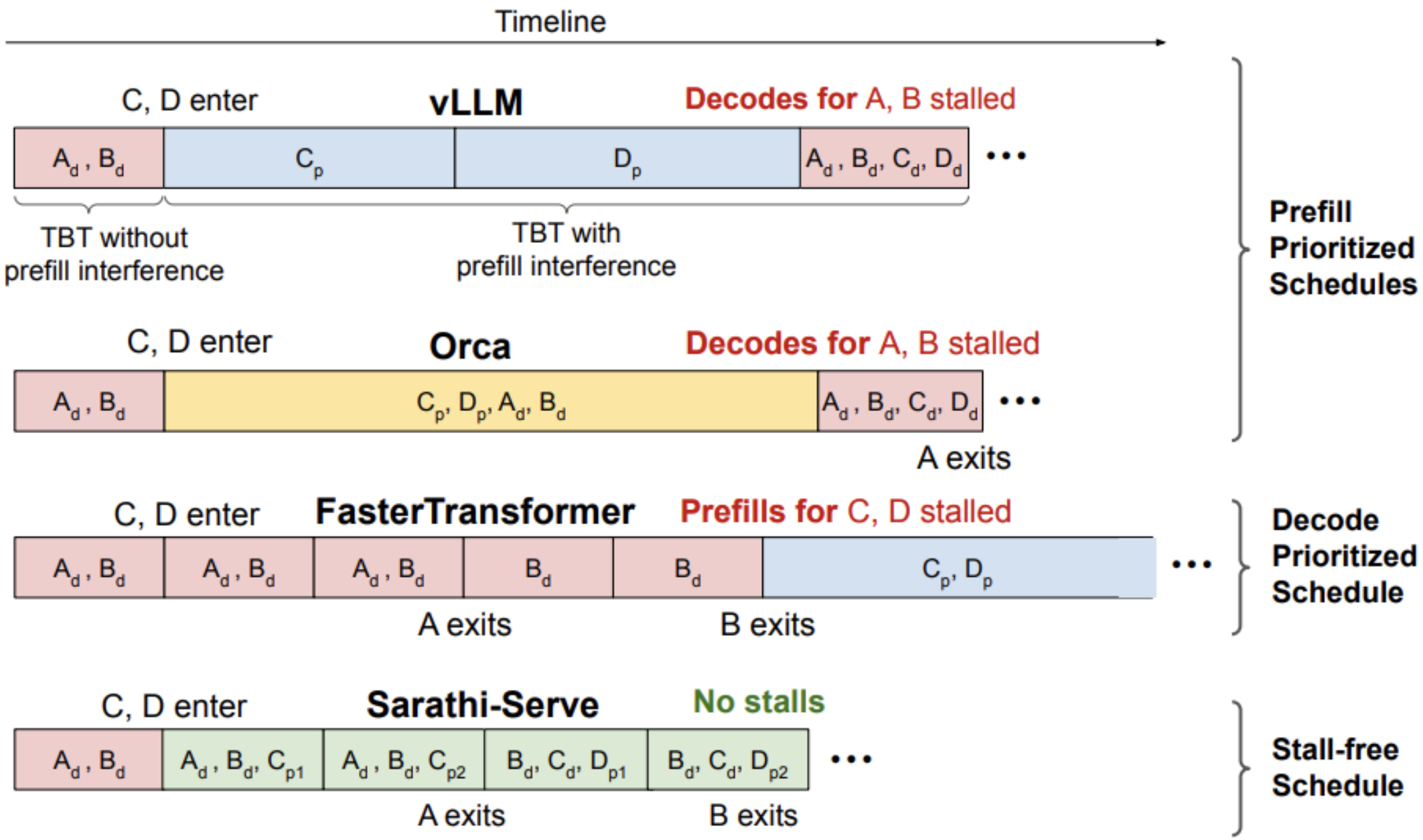
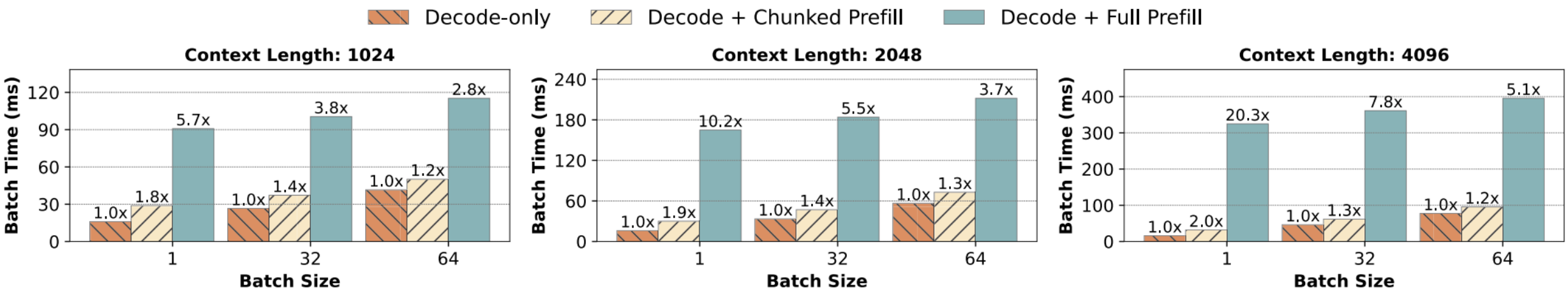
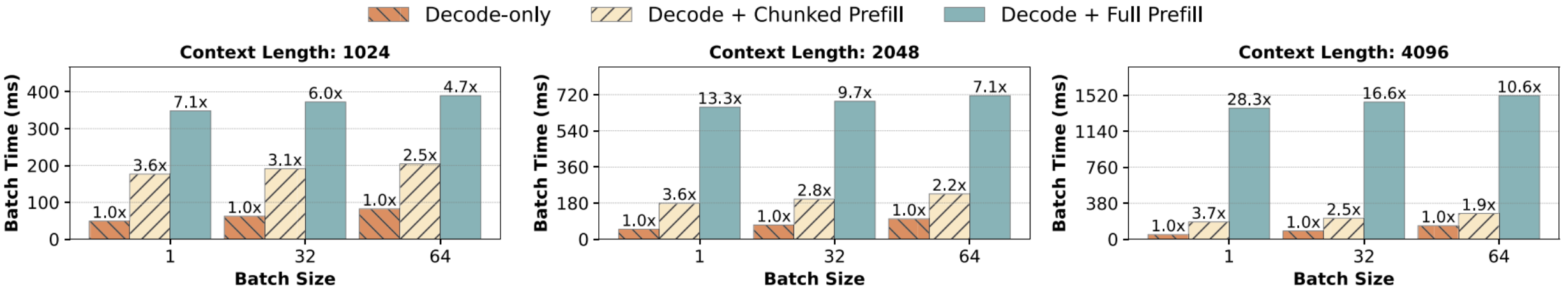


Figure 5: Arithmetic intensity trend for LLaMA2-70B linear operations with different number of token running on four A100s. Decode batches have low arithmetic intensity *i.e.,* they are bottlenecked by memory fetch time, leading to low compute utilization. Prefill batches are compute bound with sub-optimal bandwidth utilization. Sarathi-Serve forms balanced batches by combining decodes and prefill chunks to maximize both compute and bandwidth utilization.


Timeline
C, D enter
vLLM
Decodes for A, B stalled
A_d, B_d
C_p
D_p
A_d, B_d, C_d, D_d
TBT without prefill interference
TBT with prefill interference
C, D enter
Orca
Decodes for A, B stalled
A_d, B_d
C_p, D_p, A_d, B_d
A_d, B_d, C_d, D_d
A exits
C, D enter
FasterTransformer
Prefills for C, D stalled
A_d, B_d
A_d, B_d
A_d, B_d
B_d
B_d
C_p, D_p
A exits
B exits
C, D enter
Sarathi-Serve
No stalls
A_d, B_d
A_d, B_d, C_{p1}
A_d, B_d, C_{p2}
B_d, C_d, D_{p1}
B_d, C_d, D_{p2}
A exits
B exits
Prefill Prioritized Schedules
Decode Prioritized Schedule
Stall-free Schedule



(a) Mistral-7B on one A100s with token budget of 256.



(b) LLaMA2-70B on four A100s with token budget of 512.

```
Input: T_max, Application TBT SLO.
Initialize token_budget, τ ← compute_token_buget(T_max)
Initialize batch_num_tokens, n_t ← 0
Initialize current batch B ← ∅
while True do
    for R in B do
        if is_prefill_complete(R) then
            n_t ← n_t + 1
    for R in B do
        if not is_prefill_complete(R) then
            c ← get_next_chunk_size(R, τ, n_t)
            n_t ← n_t + c
    R_new ← get_next_request()
    while can_allocate_request(R_new) ∧ n_t < τ do
        c ← get_next_chunk_size(R_new, τ, n_t)
        if c > 0 then
            n_t ← n_t + c
            B ← R_new
        else
            break

    process_hybrid_batch(B)
    B ← filter_finished_requests(B)
    n_t ← 0
```

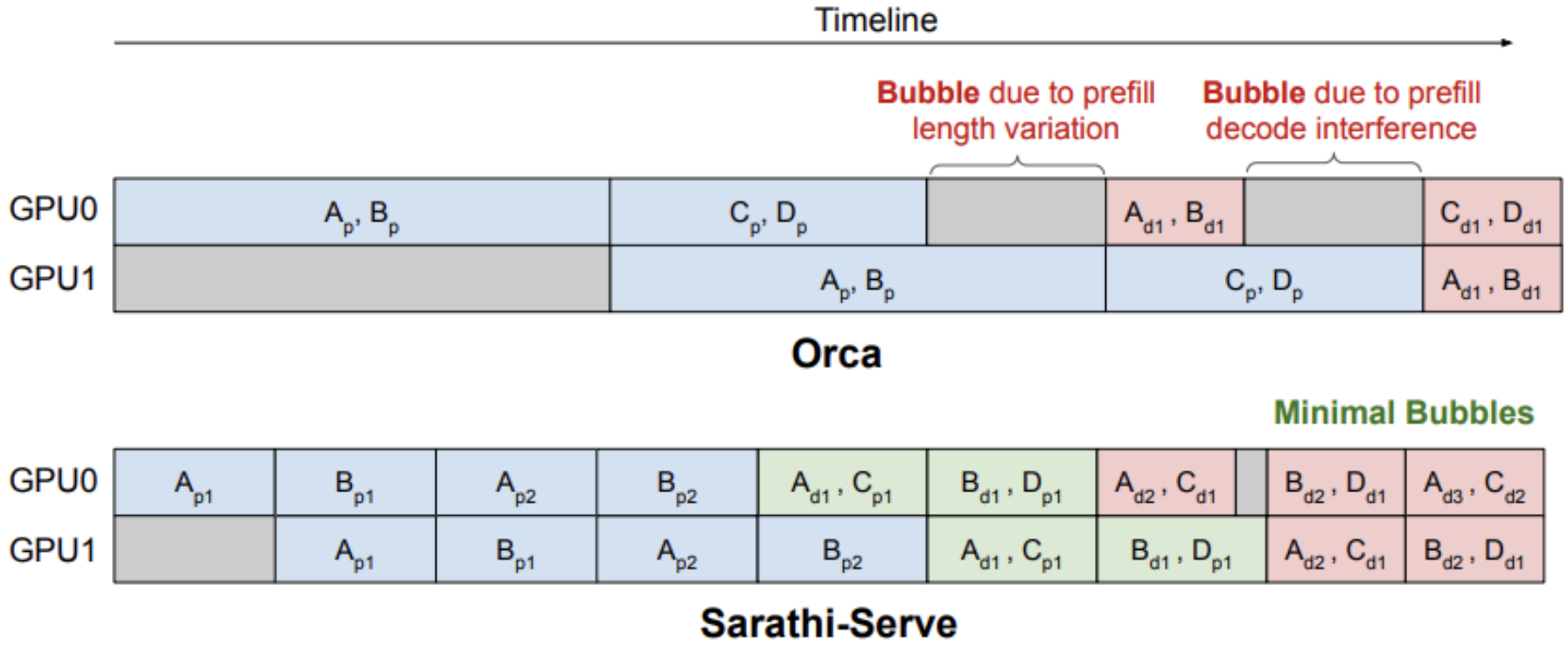


Figure 8: A 2-way pipeline parallel iteration-level schedule in Orca across 4 requests (A,B,C,D) shows the existence of pipeline bubbles due to non-uniform batch execution times. Sarathi-Serve is able to minimize these stalls by creating uniform-compute batches.